

Setup Keycloak with Spring Boot

```
keycloak:
  base-url: ${KEYCLOAK_BASE_URL:http://localhost:9090}
  realm: ${KEYCLOAK_REALM:room-system}
  admin:
    client-id: ${KEYCLOAK_ADMIN_CLIENT_ID:user-service}
    client-secret: ${KEYCLOAK_ADMIN_CLIENT_SECRET:IbJpwCh9ncqQaYd9WqVwohGDpFZoFqfB}
  client:
    client-id: ${KEYCLOAK_LOGIN_CLIENT_ID:user-login}
    client-secret: ${KEYCLOAK_LOGIN_CLIENT_SECRET:XKYp5lx6AhIEE2ZkhY9C6P5aSjHTAYbF}
```

The screenshot shows the Keycloak Admin Console interface. On the left is a dark sidebar with a navigation menu. The top of the sidebar has a dropdown menu set to 'room-system'. Below it are links for 'Manage', 'Clients', 'Client scopes', 'Realm roles', 'Users', 'Groups', 'Sessions', 'Events', 'Configure', 'Realm settings', 'Authentication', 'Identity providers', and 'User federation'. The 'Clients' link is highlighted. The main content area is titled 'Clients > Client details' and shows the details for the 'user-login' client. The client is of type 'OpenID Connect'. Below this, there is a description: 'Clients are applications and services that can request authentication of a user.' There are several tabs: 'Settings', 'Keys', 'Credentials' (which is active), 'Roles', 'Client scopes', 'Sessions', and 'Advanced'. Under the 'Credentials' tab, there are three sections: 1. 'Client Authenticator' with a dropdown menu set to 'Client Id and Secret' and a 'Save' button. 2. 'Client Secret' with a text field containing 'XKYp5lx6AhIEE2ZkhY9C6P5aSjHTAYbF', a copy icon, a delete icon, and a 'Regenerate' button. 3. 'Registration access token' with a text field, a copy icon, and a 'Regenerate' button.

The screenshot shows the Keycloak Admin Console interface, similar to the previous one, but for the 'user-service' client. The sidebar is the same, with 'Clients' highlighted. The main content area is titled 'Clients > Client details' and shows the details for the 'user-service' client. The client is of type 'OpenID Connect'. Below this, there is a description: 'Clients are applications and services that can request authentication of a user.' There are several tabs: 'Settings', 'Keys', 'Credentials' (which is active), 'Roles', 'Client scopes', 'Service accounts roles', 'Sessions', and 'Advanced'. Under the 'Credentials' tab, there are three sections: 1. 'Client Authenticator' with a dropdown menu set to 'Client Id and Secret' and a 'Save' button. 2. 'Client Secret' with a text field containing 'IbJpwCh9ncqQaYd9WqVwohGDpFZoFqfB', a copy icon, a delete icon, and a 'Regenerate' button. 3. 'Registration access token' with a text field, a copy icon, and a 'Regenerate' button.

The screenshot is showing a **typical Keycloak setup** for a Spring Boot application. The three names:

- **room-system**
- **user-login**
- **user-service**

all have different jobs. Let's explain them with a simple real-world example.

Think of Keycloak as a Hotel Reception

Imagine you have a hotel.

- **Keycloak** = the receptionist
- **Users** = hotel guests
- **Spring Boot application** = the hotel rooms

The receptionist decides:

- Who is allowed to enter
- Who they are
- What they can do

1. room-system (Realm)

This is the most important concept.

room-system is the **Realm**.

A Realm is like a **company** or a **project**.

It contains:

- Users
- Passwords
- Roles
- Clients
- Groups

For example

```
Keycloak
|
+-- room-system (Realm)
    |
    +-- Users
    +-- Roles
    +-- Clients
```



In your configuration

```
</> YAML
```



```
realm: room-system
```

means

"Everything in this application belongs to the realm called **room-system**."

Imagine you have another project.

```
school-system  
bank-system  
hospital-system
```



Each project can have its own Realm.

So:

```
Realm = One application/project/company
```



2. user-login (Client)

Inside a Realm, there can be many applications.

Each application is called a **Client**.

Your screenshot shows

```
user-login
```



This is the client that users log into.

Think of it as

```
Browser  
|  
| Login  
v  
Keycloak
```



Spring Boot redirects users to Keycloak.

Keycloak asks

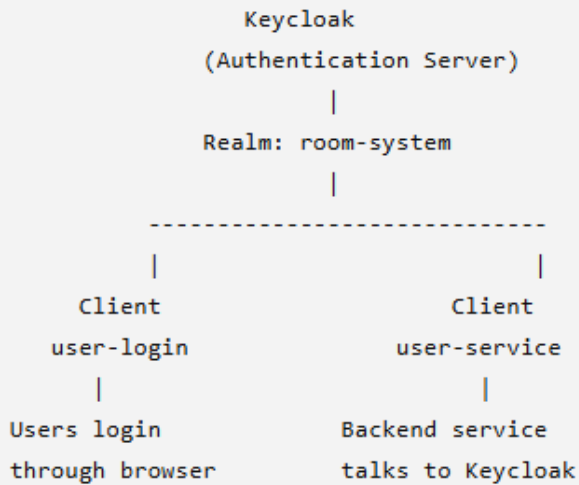
```
Username:  
Password:
```



If successful,

Keycloak returns a token.

Overall flow



Real Example

Suppose you're building a **Hotel Room Booking System**.

```
Realm
-----
hotel-system
```

The diagram shows a 'Realm' with a dashed line below it, followed by the text 'hotel-system'. A copy icon is in the top right corner.

Inside it:

```
Clients

hotel-web
hotel-mobile
booking-service
payment-service
admin-service
```

The diagram shows 'Clients' with a list of services: 'hotel-web', 'hotel-mobile', 'booking-service', 'payment-service', and 'admin-service'. A copy icon is in the top right corner.

- **hotel-web** → Website login
- **hotel-mobile** → Mobile app login
- **booking-service** → Backend API
- **payment-service** → Payment API
- **admin-service** → Admin backend

All belong to the same Realm because they're part of the same system.

In your project

- `room-system` → The Keycloak **Realm** that contains everything for your application.
- `user-login` → The **client** used when users log in to the application and receive access tokens.
- `user-service` → The **client** used by your Spring Boot backend (or administrative/service logic) to authenticate itself with Keycloak when communicating as a service rather than as a user.

So, in one sentence:

`room-system` is the container (Realm), `user-login` is the application that users log into, and `user-service` is the backend service that securely communicates with Keycloak.